

Trees — Fundamentals, Traversals and the Tree Patterns

What this is. A complete study guide for **binary trees**, built from a guided reasoning-first session (terminology, recursion, traversals, BFS, structural types, and nine problem-solving patterns through mixed/hard problems) and **extended to cover every problem the session named but did not solve**, plus implementations, complexity notes and a full recognition system.

The one idea to keep. Every tree question boils down to two questions: **what should `dfs(node)` mean**, and **what should it return?** Once those are answered, ask whether information flows **child** → **parent** (bottom-up) or **parent** → **child** (top-down), and whether the final answer is the value `dfs(root)` returns or a **separate global answer** updated along the way. Everything else is detail.

What the original session covered — and what this guide adds

Topic	Original session	This guide
Terminology (root, parent/child, siblings, leaf, edge, depth vs height, subtree)	✓	✓ (+ diagrams)
Binary tree vs BST, <code>TreeNode</code> representation	✓	✓
Why recursion fits trees; <code>dfs(node)</code> meaning; base case derivation	✓	✓
Preorder / inorder / postorder, recursive + iterative, call stack ↔ explicit stack	✓	✓ (+ all three iterative forms, tested)
BFS / level order, <code>level_size</code>	✓	✓ (+ Right Side View, Zigzag, Average of Levels, Min Depth, Maximum Width)
Tree types: full, complete, perfect, balanced, skewed	✓	✓ (+ runnable checkers for each)
Pattern 1 — simple bottom-up DFS (height, max value)	✓	✓ (+ Same Tree, Invert Tree, Subtree of Another Tree, Longest Univalued Path, Good Nodes)
Pattern 2 — top-down DFS (path sum)	✓	✓ (+ Path Sum II, Sum Root-to-Leaf Numbers, the bottom-up "twin" problem)
Pattern 3 — path + global answer (diameter, max path sum)	✓ (diameter derivation had a +1 / +2 mix-up, resolved in-chat)	✓ (+ leaf-to-leaf diameter variant, corrected explanation)
Pattern 4 — BST (search, validate, kth smallest)	✓	✓ (+ insert, delete, two-sum-BST, trim, balance, BST iterator)
Pattern 5 — LCA (binary tree + BST), distance between nodes	✓	✓ (+ LCA with parent pointers)
Pattern 6 — root-to-node path, path between two nodes	✓	✓ (+ list-of-ancestors helper)
Tree construction (preorder+inorder, inorder+postorder)	✓	✓ (+ preorder+postorder for full binary trees, sorted-array-to-BST)

Topic	Original session	This guide
Serialization / deserialization	✓	✓ (+ iterative and BFS-based codecs)
Tree DP (House Robber III, sketched Binary Tree Cameras)	✓	✓ full Binary Tree Cameras solution + Largest BST Subtree
Mixed/hard problems (Maximum Width, BFS-vs-DFS, path-sum-anywhere + hashmap, top-k + heap)	✓ (concepts, some code)	✓ all solved and tested , plus Vertical Order and Boundary Traversal as further combination examples
Templates, recognition table, complexity rule, problem-solving checklist	✓	✓

Over 60 problems and implementations are solved in Python. Every one is checked against a brute-force or independently-derived reference on hundreds of random trees, and every step table in this guide is produced by actually running the code.

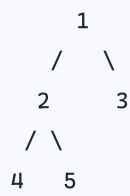
Corrections and clarifications from the session. (1) The diameter derivation initially wrote `left + 1 + right` using edge-based heights and then corrected itself mid-conversation to `left + right + 2` — this guide explains the `+2` once, cleanly, in §6.1, instead of leaving both versions in the text. (2) `dfs(None) = 0` was an initial guess for height; the session correctly derives `dfs(None) = -1` for edge-based height — this guide states the convention once and uses it consistently. (3) The session's Binary Tree Cameras sketch is correct but terse; §12.2 adds the full reasoning for the three states. (4) The `count_nodes` "quick question" at the end of §1 (page 28–29) was left unanswered in the chat ("not needed, I get it"); this guide answers it in §1.7 for completeness, since the same template reappears constantly.

Part 1 · Tree Fundamentals

The important thing is not learning to draw a tree. The important question is: why does recursion fit trees so naturally, and what should `dfs(node)` mean?*

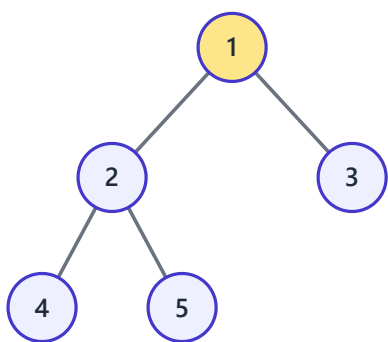
1.1 What is a tree?

A tree is a collection of nodes connected by edges, with a hierarchical structure and no cycles.



1.2 Vocabulary

Tree vocabulary: root, parent/child, siblings, leaf, edge, depth, height



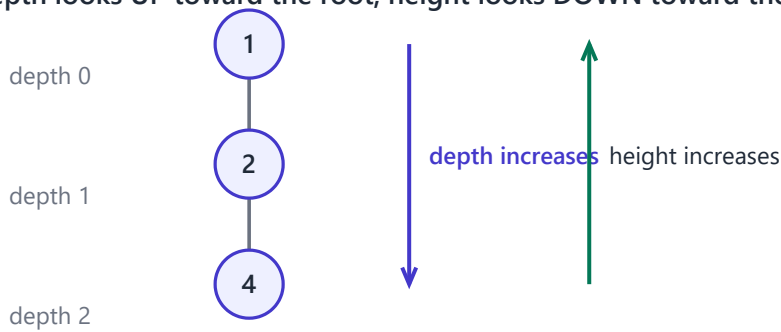
- root: 1 (no parent)
- parent of 2, 3: 1
- siblings: 2 and 3
- leaves (no children): 4, 5, 3
- edge: the line between two nodes
- depth(4) = 2 (edges up to the root)
- height(2) = 1 (edges down to its deepest leaf)

Term	Meaning
Node	Each value in the tree (here, 1, 2, 3, 4, 5)
Root	The node at the top — there is exactly one in a normal tree
Parent / child	1 is the parent of 2 and 3 ; they are its children
Siblings	Nodes sharing a parent (2 and 3)
Leaf	A node with no children (4 , 5 , 3)
Edge	The connection between two nodes
Subtree	The tree formed by a node and everything below it

1.3 Depth vs height

This distinction appears constantly in tree problems.

Depth looks UP toward the root; height looks DOWN toward the deepest leaf



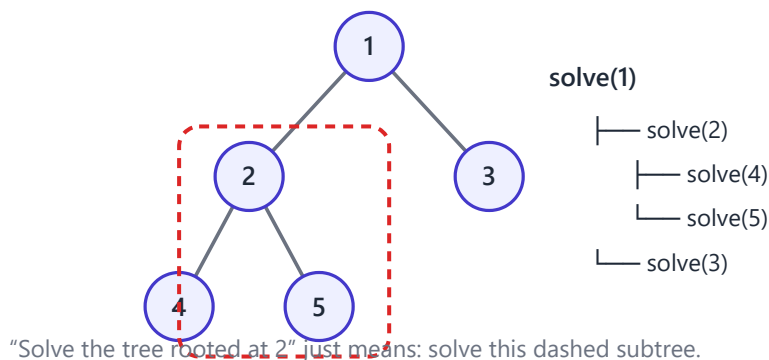
$\text{height}(1) = 2$, $\text{height}(2) = 1$, $\text{height}(4) = 0$ (edges to the deepest descendant)

- **Depth(node)** — how far the node is from the root, in edges. Depth **looks upward**.
- **Height(node)** — how far the node can reach downward to its deepest descendant, in edges. Height **looks downward**.

For our example: $\text{depth}(1)=0$, $\text{depth}(2)=1$, $\text{depth}(4)=2$; $\text{height}(1)=2$, $\text{height}(2)=1$, $\text{height}(4)=0$.

1.4 A tree is made of smaller trees

A tree is made of smaller trees: the subtree rooted at 2



Solving the problem for the subtree rooted at 2 simply means solving it for the dashed region. **Every node can be viewed as the root of its own smaller tree** — that is the recursive structure everything else builds on:

`solve(1)` — `solve(2)` | — `solve(4)` | — `solve(5)` — `solve(3)`

The question to start asking now, before any code: **if I call `solve(node)`, what exactly should that function mean?**

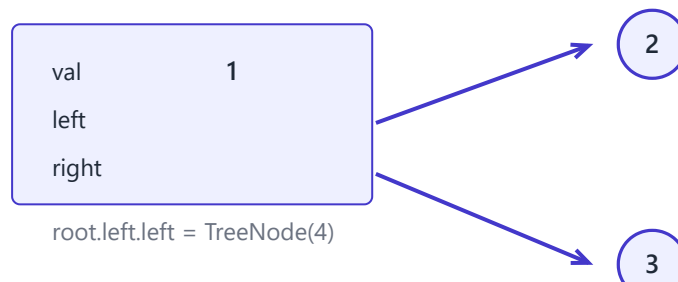
1.5 Binary trees and `TreeNode`

A **binary tree** is a tree where every node has **at most two** children — a *left child* and a *right child*. It says nothing about the values.

Binary tree $\neq$ BST. A binary tree only constrains the *shape* (≤ 2 children). A **binary search tree (BST)** additionally requires an ordering: every value in a node's left subtree is smaller than the node, and every value in its right subtree is larger. Binary tree = structure only. BST = structure + ordering invariant. We exploit the ordering starting in Part 7.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

A `TreeNode`: a value plus two pointers to its children



`root.val` asks "what value is here?"; `root.left` / `root.right` ask "which node is my left/right child?"; a missing child is `None`.

A small helper used throughout this guide, for building a tree from the LeetCode-style level-order list (`None` for a missing child) and back:

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def build_tree_list(vals):
    """LeetCode-style level-order list with None for a missing child -> a TreeNode tree."""
    if not vals or vals[0] is None:
        return None
    root = TreeNode(vals[0])
    q = deque([root])
    i = 1
    while q and i < len(vals):
        node = q.popleft()
        if i < len(vals) and vals[i] is not None:
            node.left = TreeNode(vals[i]); q.append(node.left)
        i += 1
        if i < len(vals) and vals[i] is not None:
            node.right = TreeNode(vals[i]); q.append(node.right)
        i += 1
    return root

```

1.6 Why recursion fits trees

Node 1's left subtree is rooted at 2, and 2's own subtree is {2, 4, 5} — 2 is simultaneously a *leaf of the recursion* and the *root of its own tree*. So instead of asking "how do I process all these nodes?", ask **"how do I solve the tree rooted at this one node?"**:

```

solve(node)
├─ solve(node.left)
├─ solve(node.right)
└─ combine their results

```

This gives the question you will ask for **every** tree problem in this guide: **what should `dfs(node)` return?**

1.7 Deriving the base case: height

Suppose `dfs(node)` means *the height of the subtree rooted at `node`, in edges*. What should `dfs(node)` depend on, and what should `dfs(None)` be?

Node 1 needs the heights of 2 and 3 to compute its own: $h(1) = 1 + \max(h(2), h(3))$. For a leaf like 4, both children are `None`, and we want $h(4) = 0$. Working backward from $1 + \max(h(\text{None}), h(\text{None})) = 0$, we need $h(\text{None}) = -1$.

The base case is derived from the recurrence, not memorised. This is the reasoning style used for every `dfs` in this guide: define what the function *means*, then work out what the empty case must be for that meaning to hold.

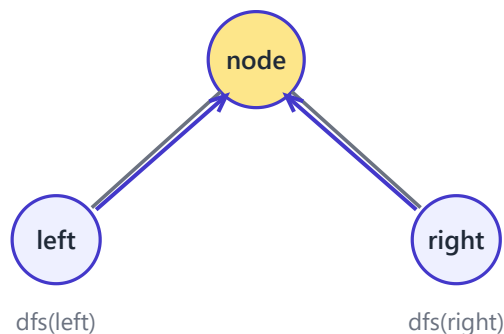
```
def dfs(node):
    if not node:
        return -1          # BASE CASE, derived above
    left = dfs(node.left)
    right = dfs(node.right)
    return 1 + max(left, right)
```

Dry run on the running example (1 over 2, 3 , with 2 over 4, 5):

call (indented by depth)	returns
None	-1
None	-1
4	0
None	-1
None	-1
5	0
2	1
None	-1
None	-1
3	0
1	2

The three conceptual steps, for *any* bottom-up problem: (1) ask the left child, (2) ask the right child, (3) combine and return to the parent.

Bottom-up DFS: children answer first, the parent combines their answers



1. ask left child
2. ask right child

3. combine(left, right)
4. return to parent

```
def dfs(node):
    if not node:
        return BASE_CASE
    left = dfs(node.left)
    right = dfs(node.right)
    return COMBINE(left, right)
```

We will reuse this template for height, node count, sum, balance, diameter, maximum path sum and House Robber III — **the only thing that changes is what `dfs(node)` means, what it returns, and how left/right are combined.**

As a quick check of the template before moving on: if `dfs(node)` means *the number of nodes in the subtree rooted at node*, the same reasoning gives `dfs(None) = 0` and `dfs(node) = 1 + dfs(left) + dfs(right)` (the `+1` counts `node` itself, unlike height which counts edges).

```
def height(node):
    """height(node) = edges to the deepest descendant. height(None) = -1 so a leaf gives 0."""
    if not node:
        return -1
    return 1 + max(height(node.left), height(node.right))

def count_nodes(node):
    """dfs(node) = number of nodes in the subtree rooted at node."""
    if not node:
        return 0
    return 1 + count_nodes(node.left) + count_nodes(node.right)

def sum_nodes(node):
    if not node:
        return 0
    return node.val + sum_nodes(node.left) + sum_nodes(node.right)
```

1.8 What can go wrong — quick facts

Mistake	Fix
Using <code>dfs(None) = 0</code> for edge-based height	Derive it from the recurrence: it must be <code>-1</code> so a leaf gives <code>0</code>
Confusing depth (looks up) and height (looks down)	Depth is a property of <i>the root's view of a node</i> ; height is a property of <i>a node's view of its descendants</i>
Forgetting a tree problem can return more than a number	<code>dfs(node)</code> can return a boolean, a node, a tuple — whatever the parent needs
Assuming recursive DFS is always $O(\log n)$ space	It's $O(h)$; a skewed tree makes $h = O(n)$ (see §14)

Part 2 · Tree Traversals

Recognition. *In what order do I need to process the nodes?*

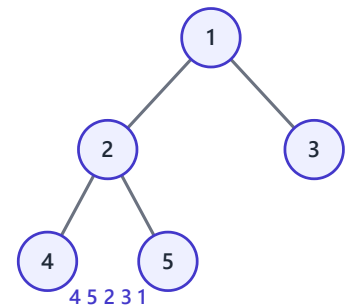
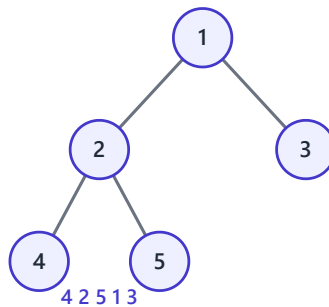
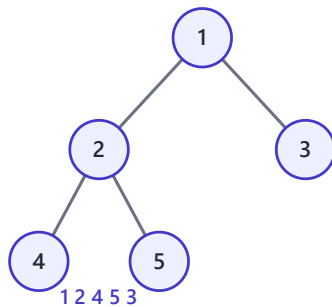
2.1 The three DFS orders

Preorder, inorder, postorder: only WHEN the current node is processed changes

PREORDER = me → left → right

INORDER = left → me → right

POSTORDER = left → right → me



Don't think of preorder/inorder/postorder as three separate pieces of code — they are **three positions for**

PROCESS(node) :

PREORDER = ME → left → right

"I'm arriving here, so deal with it now."

INORDER = left → ME → right

"Handle my left side, then me, then my right side."

POSTORDER = left → right → ME

"Finish my children before I deal with myself."

```
def preorder(root):
    """ROOT -> LEFT -> RIGHT: process the node before its children."""
    if not root:
        return []
    return [root.val] + preorder(root.left) + preorder(root.right)

def inorder(root):
    """LEFT -> ROOT -> RIGHT: process the node between its children."""
    if not root:
        return []
    return inorder(root.left) + [root.val] + inorder(root.right)

def postorder(root):
    """LEFT -> RIGHT -> ROOT: process the node after both children."""
    if not root:
        return []
    return postorder(root.left) + postorder(root.right) + [root.val]
```

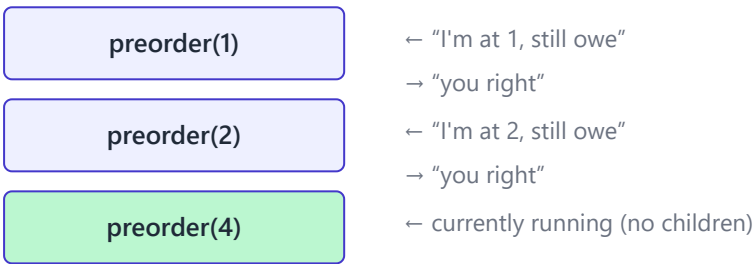
Why this matters for problem-solving. If a problem says "do something before touching any descendants" → preorder. If it says "children must be handled before the parent" → postorder. If a BST problem has a special `left` → `node` → `right` relationship → inorder (Part 7).

The connection to §1.7. The bottom-up template — ask left, ask right, then combine — processes the current node **after** both children return. That is postorder. "Postorder isn't just a traversal order — it naturally appears whenever the parent needs information computed by its children."

2.2 Recursion is an implicit stack

When `preorder(1)` calls `preorder(2)`, Python must remember "I was at 1, and I still owe a call to `preorder(node.right)`" — that bookkeeping lives on the **call stack**.

Recursion IS a stack: `preorder(2)` is on hold while `preorder(4)` runs



When `preorder(4)` returns, we pop back to `preorder(2)` and continue with its right child.

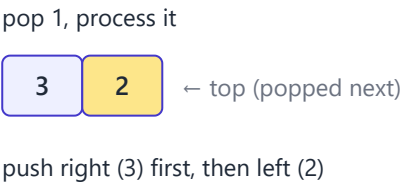
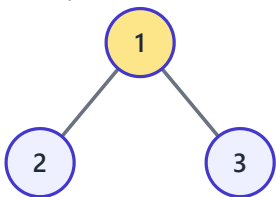
Writing iterative DFS just means maintaining that stack **ourselves**.

2.3 Iterative preorder — the push-order rule

We want `node` → `left` → `right`, and a stack is LIFO. So the child we want processed **later** must be pushed **first**:

Iterative preorder: push the **LATER** child first (stack is LIFO)

want: 1 → 2 → 3 so push 3 THEN 2, so 2 comes off first



Rule: push children in the opposite order of how you want to process them. For preorder (`left` before `right`), push `right` then `left`.

stack before	pop & process	pushed (right, then left)	stack after
[1]	1	3, 2	[3, 2]
[3, 2]	2	5, 4	[3, 5, 4]

stack before	pop & process	pushed (right, then left)	stack after
[3, 5, 4]	4	-	[3, 5]
[3, 5]	5	-	[3]
[3]	3	-	[]

```
def preorder_iterative(root):
    """Push the LATER child first: for node -> left -> right, push right then left."""
    if not root:
        return []
    stack, out = [root], []
    while stack:
        node = stack.pop()
        out.append(node.val)
        if node.right:
            stack.append(node.right)
        if node.left:
            stack.append(node.left)
    return out
```

Inorder and postorder have their own iterative forms — inorder walks all the way left before processing, postorder can be built as a reversed "root → right → left" preorder:

```
def inorder_iterative(root):
    """Walk all the way left, process, then step into the right subtree."""
    stack, out, node = [], [], root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        out.append(node.val)
        node = node.right
    return out

def postorder_iterative(root):
    """Preorder in ROOT -> RIGHT -> LEFT order, then reverse -> LEFT -> RIGHT -> ROOT."""
    if not root:
        return []
    stack, out = [root], []
    while stack:
        node = stack.pop()
        out.append(node.val)
        if node.left:
            stack.append(node.left)
        if node.right:
            stack.append(node.right)
    return out[::-1]
```

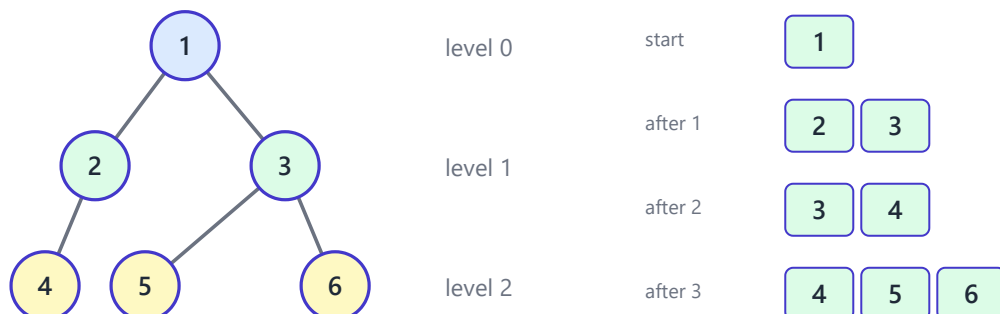
2.4 BFS / Level Order

DFS goes as deep as possible before backing up. Sometimes we want to process the tree **level by level** instead:

```
DFS:          1 → 2 → 4 → 5 → 3
Level order:  1 | 2 3 | 4 5
```

The natural structure is a **queue**: FIFO means we finish everything already waiting before we touch anything just discovered.

Level order: a queue naturally groups nodes by distance from the root



To separate levels, record the queue's size **before** processing that level — the same `level_size` idea used throughout the Queue guide:

```
from collections import deque
queue = deque([root])
while queue:
    level_size = len(queue)
    for _ in range(level_size):
        node = queue.popleft()
        # process node
        if node.left: queue.append(node.left)
        if node.right: queue.append(node.right)
```

level	queue at the start	level_size	collected
0	[1]	1	[1]
1	[2, 3]	2	[2, 3]
2	[4, 5]	2	[4, 5]

```
def level_order(root):
    """level_size = len(queue) freezes the boundary between one level and the next."""
    if not root:
        return []
    result, q = [], deque([root])
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft()
            level.append(node.val)
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)
        result.append(level)
    return result
```

Recognition signal. *Level by level, same depth, nearest node, minimum number of edges, nodes at depth K, process each level* → **BFS + queue**.

2.5 Variants that only change what you do per level

Problem	Per level, do this
Right Side View (199)	keep the node at index <code>level_size - 1</code> — the last one processed
Average of Levels (637)	sum the level, divide by <code>level_size</code>
Minimum Depth (111)	return the depth of the first leaf met (BFS beats DFS here — no need to explore deeper branches)

```
def right_side_view(root):
    """The LAST node processed in each level is the one visible from the right."""
    if not root:
        return []
    view, q = [], deque([root])
    while q:
        size = len(q)
        for i in range(size):
            node = q.popleft()
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)
            if i == size - 1:
                view.append(node.val)
    return view
```

2.6 Maximum Width of Binary Tree (LeetCode 662)

Width = the distance between the leftmost and rightmost **non-null** nodes at a level, counting the positions of any missing nodes between them.

Ordinary BFS only tells us how many *actual* nodes exist at a level — not their conceptual position in a complete-tree layout. So we attach an **index** to every node, as if the tree were stored as a complete-tree array: for a node at index i , its children sit at $2i+1$ and $2i+2$.

level (value @ index)	first index	last index	width
[(1, 0)]	0	0	1
[(3, 1), (2, 2)]	1	2	2
[(5, 3), (3, 4), (9, 6)]	3	6	4

```

def width_of_binary_tree(root):
    """Give every node the index it would have in a complete-tree array layout (2i+1, 2i+2);
    width = last index - first index + 1 for the widest level. Missing nodes never get created."""
    if not root:
        return 0
    q = deque([(root, 0)])
    answer = 0
    while q:
        first_index = q[0][1]
        last_index = first_index
        for _ in range(len(q)):
            node, index = q.popleft()
            last_index = index
            if node.left:
                q.append((node.left, 2 * index + 1))
            if node.right:
                q.append((node.right, 2 * index + 2))
        answer = max(answer, last_index - first_index + 1)
    return answer

```

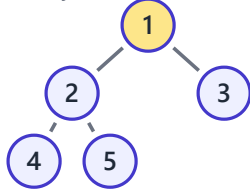
This is a good early example of the guide's central lesson: **don't blindly reach for "BFS"** — ask what extra state the queue needs to carry. Here that state is *positional*, not just *the node*.

Part 3 · Structural Types of Binary Trees

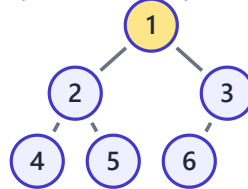
These terms describe **shape**, never values.

Structural types: what each one restricts

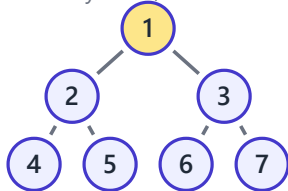
Full: every node has 0 or 2 children



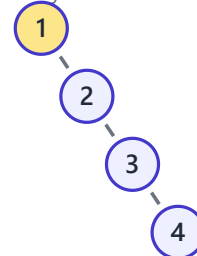
Complete: last level packed left, no gaps



Perfect: fully filled, all leaves at the same depth



Skewed: essentially a linked list



Type	Condition
Binary	at most 2 children
Full	every node has 0 or 2 children (never exactly 1)
Complete	every level full except possibly the last, and the last level packed left to right
Perfect	every internal node has 2 children <i>and</i> every leaf is at the same depth
Balanced	left/right subtree heights never differ by more than the problem's tolerance (§9 makes this precise)
Skewed / degenerate	every node has at most one child — effectively a linked list

A **perfect** tree is also **complete** and **full**. But **complete** $\neq$ **full**: 1 with only a left child 2 is complete (the last level is packed left) but not full (node 1 has exactly one child). This is one of the most common interview distinctions.

Why bother classifying at all?

1. **It changes what you can assume.** "Given a complete binary tree" tells you far more than "given a binary tree" — enough, for instance, to store it compactly in an array with no pointers (which is exactly why binary heaps are complete trees).
2. **It affects complexity.** A balanced/complete-ish tree has $h = O(\log n)$; a skewed one has $h = O(n)$. Any algorithm using $O(h)$ recursion space inherits that difference directly.
3. **Some algorithms depend on it.** A heap's array representation only works because it is complete.
4. **It tells you what a problem is really checking.** "Is this tree full?" is asking about children counts, not values.

But — **don't overuse these classifications.** For most problems, the five questions from §1.6/§1.7 (*what does dfs mean, what does it return, bottom-up or top-down, DFS or BFS, what invariant holds*) matter far more than naming the tree's shape. Think of the classification as **extra information about the input** that occasionally simplifies a problem, not a checklist to run on every tree you see.

Checking each property is itself a small bottom-up/top-down exercise:


```
def is_full(node):
    """Every node has 0 or 2 children (never exactly 1)."""
    if not node:
        return True
    if (node.left is None) != (node.right is None):
        return False
    return is_full(node.left) and is_full(node.right)
```

`is_complete` is a good example of *"the answer isn't purely bottom-up or top-down — it's about queue order"*: BFS the tree including `None` children, and once a `None` has been seen, every slot after it must also be `None`.

```
def is_complete(root):
    """Every level full except possibly the last, and the last level packed from the left:
    BFS the tree and once a None child is seen, every later slot must also be None."""
    if not root:
        return True
    q = deque([root])
    seen_none = False
    while q:
        node = q.popleft()
        if node is None:
            seen_none = True
        else:
            if seen_none:
                return False
            q.append(node.left)
            q.append(node.right)
    return True
```

```
def is_perfect(root):
    """Every internal node has 2 children and every leaf is at the same depth: node count == 2**
    (h+1) - 1."""
    h = height(root)
    return count_nodes(root) == 2 ** (h + 1) - 1
```

Balance needs **two** pieces of information per node (height *and* whether the subtree is already balanced) — the first example in this guide of a `dfs` that returns more than one value, a pattern developed fully in Part 9:

```
def is_balanced(root):  
    """dfs(node) returns (height, balanced) - the parent needs both. -1 also serves as an  
'unbalanced' signal  
    in the optimised version below, but this version keeps the two pieces of state explicit."""  
    def dfs(node):  
        if not node:  
            return -1, True  
        lh, lb = dfs(node.left)  
        rh, rb = dfs(node.right)  
        bal = lb and rb and abs(lh - rh) <= 1  
        return 1 + max(lh, rh), bal  
    return dfs(root)[1]
```

Part 4 · Pattern 1 — Simple Bottom-Up DFS

Recognition. Solve the problem for the current node using answers already computed by its children. Ask: **what should `dfs(node)` return?**

```
def dfs(node):
    if not node:
        return BASE_CASE
    left = dfs(node.left)
    right = dfs(node.right)
    return COMBINE(left, right)
```

4.1 Maximum Depth of Binary Tree (LeetCode 104)

Already derived fully in §1.7: `dfs(node) = 1 + max(dfs(left), dfs(right))`, `dfs(None) = -1`.

```
def max_depth(root):
    """dfs(node) = height of the subtree rooted at node, measured in edges."""
    if not root:
        return -1
    return 1 + max(max_depth(root.left), max_depth(root.right))
```

4.2 Maximum value in a binary tree

What should `dfs(node)` return, and how does the parent combine its children's answers?

The tempting first guess is `max(left, right)` — but that's missing something: **the parent's own value is also a candidate.**

```
dfs(node) = max(node.val, dfs(left), dfs(right))
```

The lesson. Don't automatically write `answer = combine(left, right)`. Ask: *what information does the parent itself contribute, beyond what its children return?*

```
def max_value(root):
    """dfs(node) = max(node.val, left, right) - the parent's OWN value is part of the answer too."""
    if not root:
        return float("-inf")
    return max(root.val, max_value(root.left), max_value(root.right))
```

4.3 Same Tree (LeetCode 100)

Now `dfs` needs information from **two** trees at once, so the recursion compares corresponding positions:

1. Both `None` → same.
2. One `None`, one not → different.
3. Values differ → different.
4. Otherwise, recurse into both left pairs and both right pairs.

```
def same_tree(p, q):  
    """Compare corresponding structure and values recursively."""  
    if not p and not q:  
        return True  
    if not p or not q or p.val != q.val:  
        return False  
    return same_tree(p.left, q.left) and same_tree(p.right, q.right)
```

4.4 Invert Binary Tree (LeetCode 226)

Bottom-up in the sense that each node's own inversion doesn't depend on its children's *values* changing — but the recursive calls still happen on the original children before we swap them:

```
def invert_tree(root):  
    """LeetCode 226: swap every node's children, bottom-up."""  
    if not root:  
        return None  
    root.left, root.right = invert_tree(root.right), invert_tree(root.left)  
    return root
```

4.5 Subtree of Another Tree (LeetCode 572)

Reuses `same_tree` as a helper: check whether `sub` matches the subtree rooted at every node of `root`.

```
def is_subtree(root, sub):  
    """LeetCode 572: is 'sub' identical to SOME subtree of 'root'?"""  
    if not root:  
        return sub is None  
    if same_tree(root, sub):  
        return True  
    return is_subtree(root.left, sub) or is_subtree(root.right, sub)
```

4.6 Longest Univalue Path (LeetCode 687)

This is the pattern's first real step up in difficulty, because it needs **two** different pieces of reasoning at the same node: a *candidate* using both sides, and a *return value* using only one.

- A side can be extended only if the **child's value matches the current node's**.
- The candidate through the node may use **both** sides (`left_path + right_path`).
- The value handed to the parent can only continue in **one** direction (`max(left_path, right_path)`), because a path returned upward cannot fork.

```
def longest_univalue_path(root):
    """LeetCode 687. A side can be extended only if the child's value matches the current node's;
    the candidate at a node may use BOTH sides, but the value returned upward uses only ONE."""
    answer = 0

    def dfs(node):
        nonlocal answer
        if not node:
            return 0
        left = dfs(node.left)
        right = dfs(node.right)
        left_path = left + 1 if node.left and node.left.val == node.val else 0
        right_path = right + 1 if node.right and node.right.val == node.val else 0
        answer = max(answer, left_path + right_path)
        return max(left_path, right_path)

    dfs(root)
    return answer
```

This structure — *candidate uses both sides, return value uses one* — is exactly the diameter pattern from Part 6, arriving early because the underlying idea (*what can I return vs what is the best answer here*) is so central it's worth seeing twice.

4.7 Count Good Nodes in Binary Tree (LeetCode 1448)

A node is "good" if no ancestor on its root-to-node path has a **greater** value. This needs to know the maximum seen **so far on the path from the root** — information that flows **parent** → **child**, which is Pattern 2 (top-down), included here because it makes a clean contrast with the bottom-up problems above:

```
def good_nodes(root):
    """LeetCode 1448: a node is 'good' if no ancestor (root to node) has a greater value.
    Top-down: carry the max value seen so far on the path."""
    def dfs(node, max_so_far):
        if not node:
            return 0
        good = 1 if node.val >= max_so_far else 0
        max_so_far = max(max_so_far, node.val)
        return good + dfs(node.left, max_so_far) + dfs(node.right, max_so_far)

    return dfs(root, float("-inf"))
```

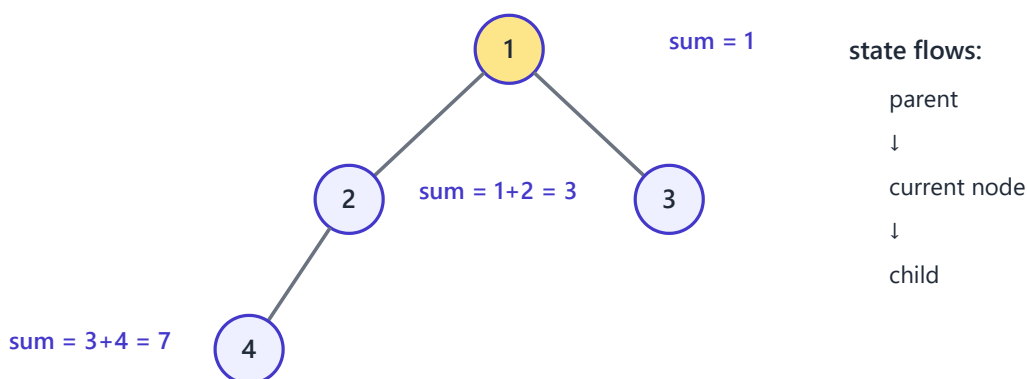
Recognition signal for Pattern 1. *Height/depth, node count, subtree sum, max/min in a subtree, whether a subtree satisfies some property* — and the answer for a node can be determined **from its children's answers alone** → bottom-up DFS, `dfs(node)` returns something.

Part 5 · Pattern 2 — Top-Down DFS

Recognition. Information now flows the **other** way: **parent** → **child**. Ask: "What does the parent need to pass down to its children?"

```
def dfs(node, state):
    if not node:
        return
    new_state = UPDATE(state, node)
    dfs(node.left, new_state)
    dfs(node.right, new_state)
```

Top-down DFS: the parent passes accumulated state **DOWN** to its children



5.1 Path Sum (LeetCode 112)

Does a root-to-leaf path sum to `target` ?

At node `2`, we need to know what happened **above** it — the accumulated sum from the root. That is state the parent must hand down.

```
def has_path_sum(root, target):
    """LeetCode 112. Carry current_sum from the root down; check it only at a leaf."""
    def dfs(node, current_sum):
        if not node:
            return False
        current_sum += node.val
        if not node.left and not node.right:
            return current_sum == target
        return dfs(node.left, current_sum) or dfs(node.right, current_sum)

    return dfs(root, 0)
```

Recognition signal. Root-to-node paths, path sum from the root, depth information, ancestor constraints, state carried along a path → top-down DFS.

5.2 Path Sum II (LeetCode 113)

Now return **every** qualifying root-to-leaf path, not just a boolean. Build the path with **backtracking**: append the node, recurse, and pop it back off once its subtree is exhausted.

```
def path_sum_all(root, target):
    """LeetCode 113: return every root-to-leaf path that sums to target. Backtrack the path list."""
    result, path = [], []

    def dfs(node, remaining):
        if not node:
            return
        path.append(node.val)
        remaining -= node.val
        if not node.left and not node.right and remaining == 0:
            result.append(list(path))
        else:
            dfs(node.left, remaining)
            dfs(node.right, remaining)
        path.pop()

    dfs(root, target)
    return result
```

5.3 Sum Root to Leaf Numbers (LeetCode 129)

Each root-to-leaf path is read as a decimal number: `new_number = old_number * 10 + node.val`.

```
def sum_numbers(root):
    """LeetCode 129. Each root-to-leaf path is a decimal number: new = old * 10 + node.val."""
    def dfs(node, current):
        if not node:
            return 0
        current = current * 10 + node.val
        if not node.left and not node.right:
            return current
        return dfs(node.left, current) + dfs(node.right, current)

    return dfs(root, 0)
```

5.4 The bottom-up twin: Maximum Root-to-Leaf Path Sum

"What is the maximum root-to-leaf path sum?" looks like the same family as Path Sum — but it isn't.

The session initially answered "carry the accumulated sum" here too, and that's a natural instinct — but it's not quite right for this question. The **parent** needs the best result each child can offer, which flows **child** → **parent**. So `dfs(node)` should mean the best sum from `node` down to any leaf in its subtree:

```
dfs(node) = node.val + max(dfs(left), dfs(right))
```

```
def max_root_to_leaf(root):  
    """The BOTTOM-UP twin of has_path_sum: dfs(node) = best root-to-leaf sum FROM node down."""  
    if not root:  
        return 0  
    if not root.left and not root.right:  
        return root.val  
    best = max(  
        max_root_to_leaf(root.left) if root.left else float("-inf"),  
        max_root_to_leaf(root.right) if root.right else float("-inf"),  
    )  
    return root.val + best
```

The skill to take from this pair of problems. Don't choose a DFS direction just because the problem mentions "path". Ask: *does the parent need something the children compute (bottom-up), or does the child need something the parent already knows (top-down)?*

	"Does a root-to-leaf path sum to X?"	"What is the max root-to-leaf sum?"
Direction	top-down	bottom-up
Carries/returns	<code>current_sum</code> down to each leaf	best sum, up from each leaf

Part 6 · Pattern 3 — Path Problems + Global Answer

Recognition. The best answer can appear anywhere in the tree, not necessarily at or through the root. This is the pattern where `dfs(node)` returning a value to the parent and the overall answer become genuinely different things.

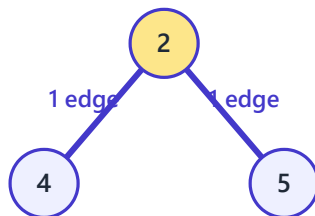
6.1 Diameter of a Binary Tree (LeetCode 543)

The diameter is the longest path between **any** two nodes (not necessarily through the root).

At node 2 with children 4, 5, the longest path *through* 2 is $4 \rightarrow 2 \rightarrow 5$. To build it, node 2 needs the longest downward path into its left subtree and into its right subtree.

Deriving the +2 (not +1). With height measured in **edges** (`dfs(None) = -1`, a leaf gives 0):

Why +2, not +1: each side contributes ITS downward edges, plus one edge into the node



left height = 0 (4 is a leaf)

right height = 0 (5 is a leaf)

path $4 \rightarrow 2 \rightarrow 5 = \text{left} + \text{right} + 2$
 $= 0 + 0 + 2 = 2 \text{ edges}$

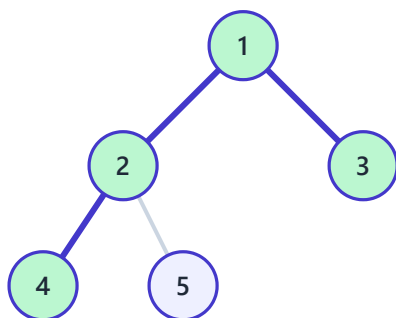
For 2 with two leaf children: `left = dfs(4) = 0`, `right = dfs(5) = 0`. Physically drawing the path $4 - 2 - 5$ shows **two edges**: one from the left side into 2, one from 2 into the right side. So:

path through node = `left_height + right_height + 2`

The +2 is *one edge connecting each side to the current node* — not "the current node itself" (which would suggest +1). Working a second example confirms it: with `left_height = 1` (via $2 \rightarrow 4 \rightarrow 6$) and `right_height = 0`, the path $6 \rightarrow 4 \rightarrow 2 \rightarrow 5$ has 3 edges, and $1 + 0 + 2 = 3 \checkmark$.

But the diameter doesn't have to pass through the root. So at *every* node we ask "could the best diameter pass through here?" and keep a **global** answer, while `dfs(node)` still returns only the height (one number) to its parent:

Diameter: `dfs(node)` returns height to the parent, but the GLOBAL answer uses BOTH sides



path $4 \rightarrow 2 \rightarrow 1 \rightarrow 3$ has 3 edges

at node 2:

left = height(4) = 0

right = height(5) = 0

path through 2 = `left + right + 2` = 2

`dfs(2)` returns to its parent only $1 + \max(\text{left}, \text{right}) = 1$

node	left height	right height	left+right+2	global best so far	
4	-1	-1	0	0	-
5	-1	-1	0	0	-
2	0	0	2	2	updated
3	-1	-1	0	2	-
1	1	0	3	3	updated

```
def dfs(node):
    if not node:
        return -1
    left = dfs(node.left)
    right = dfs(node.right)
    answer = max(answer, left + right + 2)  # candidate: BOTH sides
    return 1 + max(left, right)           # return: ONE side only
```

```
def diameter(root):
    """LeetCode 543. dfs(node) returns height (ONE side, for the parent); the global answer combines BOTH sides: left + right + 2 (edge-based height, +1 edge into each side)."""
    answer = 0

    def dfs(node):
        nonlocal answer
        if not node:
            return -1
        left = dfs(node.left)
        right = dfs(node.right)
        answer = max(answer, left + right + 2)
        return 1 + max(left, right)

    dfs(root)
    return answer
```

The pattern to internalise. `dfs(node)` returns useful information to the **parent** (one side); a separate step, at every node, updates the **global** answer (both sides). This is different from height alone, where everything the parent needs is simply the returned value.

Recognition signal. *Best answer anywhere in the tree, and a node can form a candidate from information its children return* → bottom-up DFS + **global answer**.

6.2 Variant: diameter between two leaves only

Mixed Problem: what if both endpoints of the path must be **leaves**? The naive `left + right + 2` formula is no longer automatically correct — a node with only one child that is itself a leaf shouldn't count that missing side as `-1` the way ordinary diameter does. `dfs(node)` now means *the best node-to-leaf distance, and a candidate needs a leaf reachable on both** sides*:

```
def diameter_leaf_to_leaf(root):
    """Variant: the path's two endpoints must both be LEAVES. dfs(node) returns the best
    node-to-leaf distance; a candidate at `node` needs a leaf on BOTH sides."""
    answer = 0

    def dfs(node):
        nonlocal answer
        if not node:
            return 0
        if not node.left and not node.right:
            return 0
        left = dfs(node.left) if node.left else None
        right = dfs(node.right) if node.right else None
        if left is not None and right is not None:
            answer = max(answer, left + right + 2)
            return 1 + max(left, right)
        return 1 + (left if left is not None else right)

    dfs(root)
    return answer
```

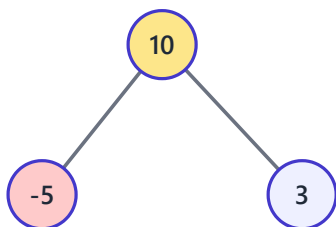
This variant exists specifically to make the point the session emphasises: **don't memorise** `diameter = left + right + 2`. Ask what your returned value actually represents, every time.

6.3 Binary Tree Maximum Path Sum (LeetCode 124)

The longest-path idea, but now with **values that can be negative**, and we want the maximum **sum**, not the longest count.

If a child's best downward contribution is negative, including it only hurts — so we clamp it: `contribution = max(0, dfs(child))`.

Maximum path sum: clamp a negative contribution to 0 before using it



`left = max(0, -5) = 0`

`right = max(0, 3) = 3`

`candidate = 0 + 10 + 3 = 13`

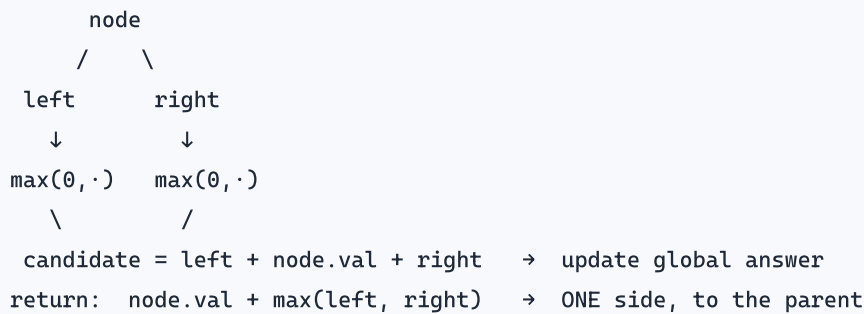
(not `-5 + 10 + 3 = 8`)

return to parent: `10 + max(0,3) = 13`

Two different questions again:

1. **What can I return to my parent?** The parent can only continue the path through **one** branch (a path can't fork and still continue upward): `node.val + max(left, right)`.

2. What is the best path passing through this node, for the global answer? Both sides are allowed here: `left + node.val + right`.



node	clamped left	clamped right	candidate	global best	
9	0	0	9	9	updated
15	0	0	15	15	updated
7	0	0	7	15	-
20	15	7	42	42	updated
-10	9	35	34	42	-

```

def max_path_sum(root):
    """LeetCode 124. A negative contribution only hurts, so clamp each side to >= 0. The global
    candidate may use BOTH sides; the value returned upward uses only ONE."""
    answer = float("-inf")

    def dfs(node):
        nonlocal answer
        if not node:
            return 0
        left = max(0, dfs(node.left))
        right = max(0, dfs(node.right))
        answer = max(answer, node.val + left + right)
        return node.val + max(left, right)

    dfs(root)
    return answer

```

Recognition signal. "Maximum sum/path between any two nodes" → (1) bottom-up DFS, (2) each child returns its best **downward** contribution, (3) negative contributions are clamped to 0, (4) the current node's candidate may use **both** sides for the global answer, (5) only **one** side is returned to the parent.

This "global answer vs. returned value" distinction is one of the most important ideas in this whole guide — it reappears in Tree DP (Part 12) and the Largest BST Subtree problem (§12.3).

Part 7 · Pattern 4 — BST Problems

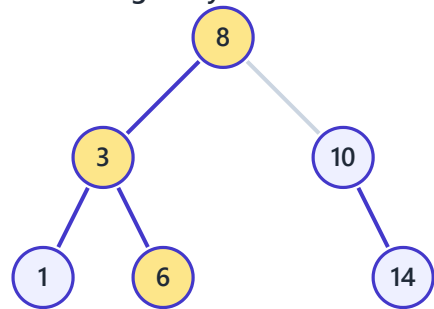
Recognition. Given a BST (or "exploit the ordering"). Unlike an ordinary binary tree, a BST's **values** tell us which direction to search.

7.1 Why the ordering matters



Searching for 7: at 8, since $7 < 8$, the entire **right** subtree can be discarded without looking at it. At 3, $7 > 3 \rightarrow$ go right. At 6, $7 > 6 \rightarrow$ go right $\rightarrow$ found.

BST ordering lets you eliminate a whole subtree at every step



searching for 7:

- $7 < 8 \rightarrow$ go left (skip 10's whole subtree)
- $7 > 3 \rightarrow$ go right
- $7 > 6 \rightarrow$ go right $\rightarrow$ not found here

one comparison eliminates an entire subtree

The core BST idea. Use the ordering to eliminate an entire subtree, rather than searching both sides the way an ordinary binary tree would require.

BST built by inserting 8, 3, 10, 1, 6, 14, 4, 7 in order.

node visited	decision
8	$7 < 8 \rightarrow$ go left
3	$7 > 3 \rightarrow$ go right
6	$7 > 6 \rightarrow$ go right
7	found

```
def search_bst(root, target):
    """The ordering rule eliminates an entire subtree at every step."""
    node = root
    while node:
        if node.val == target:
            return node
        node = node.left if target < node.val else node.right
    return None
```

Recognition signal. *BST, binary search tree, ordered binary tree, find/insert/delete, predecessor/successor, kth smallest/largest* → ask "can I exploit the ordering instead of exploring both subtrees?"

7.2 Insert and Delete

Insertion follows the same "go left or right" logic to find the empty slot:

```
def insert_into_bst(root, val):
    if not root:
        return TreeNode(val)
    if val < root.val:
        root.left = insert_into_bst(root.left, val)
    else:
        root.right = insert_into_bst(root.right, val)
    return root
```

Deletion (LeetCode 450) is the more interesting one, because removing a node with **two** children can't simply delete it — something has to take its place while preserving the ordering. The standard trick: replace its value with its **inorder successor** (the smallest node in its right subtree), then delete that successor instead (which has at most one child, so it's the easy case).

```
def delete_node_bst(root, key):
    """LeetCode 450. Deleting a node with two children: splice in its inorder successor
    (the smallest node in the right subtree), then delete that successor instead."""
    if not root:
        return None
    if key < root.val:
        root.left = delete_node_bst(root.left, key)
    elif key > root.val:
        root.right = delete_node_bst(root.right, key)
    else:
        if not root.left:
            return root.right
        if not root.right:
            return root.left
        successor = root.right
        while successor.left:
            successor = successor.left
        root.val = successor.val
        root.right = delete_node_bst(root.right, successor.val)
    return root
```

7.3 The inorder connection

Inorder traversal of a BST produces values in sorted order. (For our tree above: 1 3 4 6 7 8 10 13 14.)

This isn't a coincidence to memorise — it *falls out* of what inorder means (`left` → `me` → `right`) combined with what the BST ordering guarantees (everything in the left subtree is smaller, everything in the right is larger). So `left` → `me` → `right` on a BST is exactly `smaller` → `me` → `larger` .

Kth Smallest Element in a BST (LeetCode 230). We don't need to build the whole sorted list — stop the inorder traversal once `k` nodes have been visited:

```
def kth_smallest(root, k):
    """LeetCode 230. Inorder = sorted order for a BST; stop once we have visited k nodes."""
    stack, node = [], root
    while stack or node:
        while node:
            stack.append(node); node = node.left
        node = stack.pop()
        k -= 1
        if k == 0:
            return node.val
        node = node.right

def bst_iterator_inorder(root):
    """A generator version of inorder for a BST, used by BSTIterator-style problems (LeetCode 173)."""
    stack, node = [], root
    while stack or node:
        while node:
            stack.append(node); node = node.left
        node = stack.pop()
        yield node.val
        node = node.right
```

Recognition signal. *Kth smallest/largest in a BST, sorted order of a BST* → BST → inorder → sorted sequence → kth element.

7.4 Validate Binary Search Tree (LeetCode 98)

A common mistake: checking only `left.val < node.val < right.val`. That's **not enough** — a node must satisfy **every ancestor's** constraint, not just its immediate parent's:

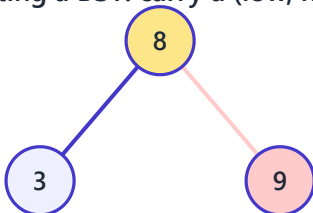
```

      8
     / \
    3   10
   / \
  1   9

```

← 9 > 3 locally looks fine, but 9 is in 8's LEFT subtree, so it must be < 8. INVALID.

Validating a BST: carry a (low, high) RANGE down, not just compare with the parent



range at 8: $(-\infty, +\infty)$

range at 3 (left of 8): $(-\infty, 8)$

range at 9 (right of 3): $(3, 8)$

9 $\notin$ (3, 8) → INVALID

(9 > 3 looks fine locally, but 9 > 8 breaks the ancestor 8)

So we carry a **range** (low, high) downward — this is *top-down DFS with state* again, but the state is a constraint rather than an accumulated value:

```
root:          (-inf, +inf)
going left:    (low, node.val)
going right:   (node.val, high)
```

node	allowed range	check
8	(-inf, inf)	OK
3	(-inf, 8)	OK
1	(-inf, 3)	OK
9	(3, 8)	VIOLATES range

```
def is_valid_bst(root):
    """LeetCode 98. NOT just left.val < node.val < right.val - every node must obey EVERY ancestor's
    bound, so the (low, high) RANGE is carried down and narrowed at each step."""
    def dfs(node, low, high):
        if not node:
            return True
        if not (low < node.val < high):
            return False
        return dfs(node.left, low, node.val) and dfs(node.right, node.val, high)

    return dfs(root, float("-inf"), float("inf"))
```

Recognition signal. *Validate BST, ancestor constraints, "must fall within a range", inherited restriction* → parent passes constraints down, child narrows them.

7.5 More BST problems built on the same two ideas

Two Sum IV — Input is a BST (LeetCode 653). Walk the tree in sorted (inorder) order, keeping a seen-set — the array-style two-sum trick, riding on the BST-gives-sorted-order fact:

```
def two_sum_bst(root, k):
    """LeetCode 653: two values in a BST that sum to k, via a seen-set while walking inorder."""
    seen = set()
    for val in bst_iterator_inorder(root):
        if k - val in seen:
            return True
        seen.add(val)
    return False
```

Trim a Binary Search Tree (LeetCode 669). The ordering tells us which whole side to discard: if the current value is below low, the entire left subtree is too small, so recurse only into the right (and vice versa).

```
def trim_bst(root, low, high):
    """LeetCode 669: keep only nodes with low <= val <= high, re-linking around removed ones."""
    if not root:
        return None
    if root.val < low:
        return trim_bst(root.right, low, high)
    if root.val > high:
        return trim_bst(root.left, low, high)
    root.left = trim_bst(root.left, low, high)
    root.right = trim_bst(root.right, low, high)
    return root
```

Balance a Binary Search Tree (LeetCode 1382) / Convert Sorted Array to BST (LeetCode 108). Inorder gives the sorted values; rebuild by always choosing the **middle** value as the root — the same idea underlies both problems, one starting from a BST and one from a plain sorted array:

```
def balance_bst(root):
    """LeetCode 1382: inorder gives the sorted values; rebuild a balanced BST by always
    picking the middle value as the root (this is also how sorted-array-to-BST works)."""
    values = inorder(root)

    def build(lo, hi):
        if lo > hi:
            return None
        mid = (lo + hi) // 2
        node = TreeNode(values[mid])
        node.left = build(lo, mid - 1)
        node.right = build(mid + 1, hi)
        return node

    return build(0, len(values) - 1)

def sorted_array_to_bst(nums):
    """LeetCode 108: the same 'pick the middle as root' idea as balance_bst, from a sorted array
    directly."""
    def build(lo, hi):
        if lo > hi:
            return None
        mid = (lo + hi) // 2
        node = TreeNode(nums[mid])
        node.left = build(lo, mid - 1)
        node.right = build(mid + 1, hi)
        return node

    return build(0, len(nums) - 1)
```

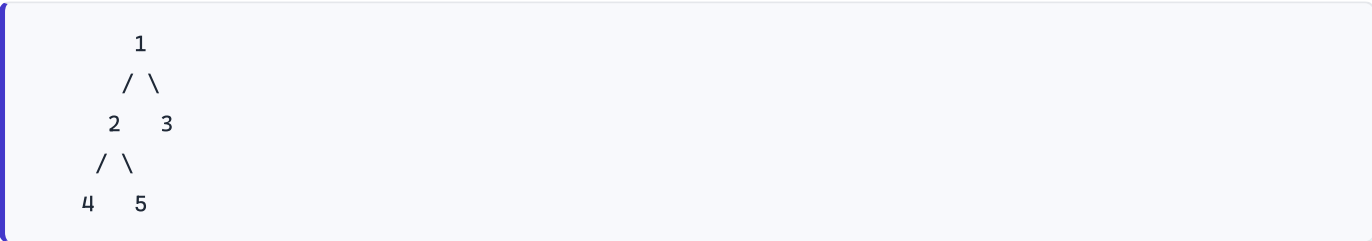
7.6 Pattern 4 summary

Problem	What the ordering buys you
Search / Insert	eliminate a whole subtree per comparison
Delete	successor replacement keeps the ordering intact
Kth Smallest	inorder = sorted, stop early
Validate BST	a <i>range</i> , not a parent comparison, must propagate down
Two Sum IV	inorder gives a sorted sequence for free
Trim	discard an entire side based on one comparison
Balance / Sorted Array → BST	pick the middle of the sorted values as the root, recursively

Part 8 · Pattern 5 — Lowest Common Ancestor (LCA)

Recognition. Given two target nodes, find the deepest node that is an ancestor of both.

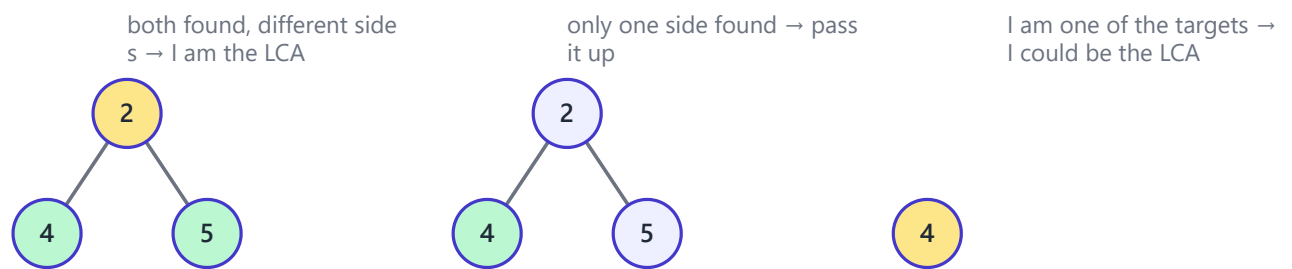
8.1 Ordinary binary tree — LeetCode 236



$LCA(4, 5) = 2$ (both live in 2's subtree). $LCA(4, 3) = 1$ (they split at the root).

At any node there are three possibilities:

LCA: three cases at every node while searching for p and q



- 1. **The current node is one of the targets** — it could be the LCA (or an ancestor of the other target further up).
- 2. **The targets are on different sides** — both children report "found something" → the current node is where the paths split → it **is** the LCA.
- 3. **Both targets are on the same side** — the other side reports nothing, so we simply pass the found side's answer up.

So `dfs(node)` returns: *the node that is the LCA of whatever targets were found in this subtree* — which could be `None`, `p`, `q`, or the actual LCA. This is another example of `dfs` returning something other than a number.

call	result
4	IS a target → return 4
5	IS a target → return 5
2	both sides found → return 2 (LCA)
3	pass up: None
1	pass up: 2

```
def lca_binary_tree(root, p, q):
    """LeetCode 236. dfs(node) returns None / p / q / the LCA - whatever this subtree can tell its
    parent.

    Both sides non-None => this node is where the paths split => LCA."""
    if not root or root is p or root is q:
        return root
    left = lca_binary_tree(root.left, p, q)
    right = lca_binary_tree(root.right, p, q)
    if left and right:
        return root
    return left if left else right
```

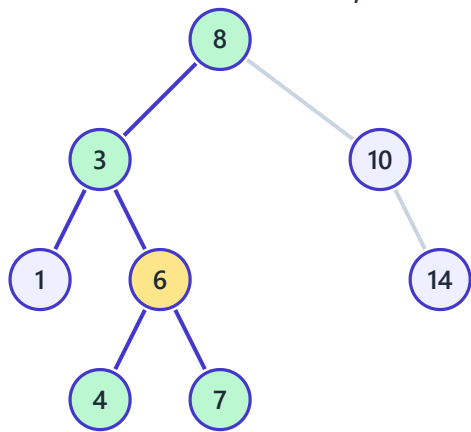
Recognition signal. *Common ancestor, lowest/deepest ancestor, relationship between two nodes → ask "where do the paths to these two nodes split?"*

8.2 LCA in a BST — LeetCode 235

The ordering lets us skip the general algorithm entirely:

```
both p, q < node    → go left
both p, q > node    → go right
one < node, one > node (or node is p or q) → node IS the LCA
```

LCA in a BST: both < node → left; both > node → right; else the split point is the LCA



```
p = 4, q = 7
at 8: both < 8 → left
at 3: both > 3 → right
at 6: 4 < 6 < 7 → split here
LCA = 6
```

node visited	decision
8	both 4,7 < 8 → left
3	both 4,7 > 3 → right
6	4 < 6 < 7 → split here

```
def lca_bst(root, p, q):
    """LeetCode 235. The ordering rule tells us the direction directly: no need to search both
    sides."""
    node = root
    while node:
        if p.val < node.val and q.val < node.val:
            node = node.left
        elif p.val > node.val and q.val > node.val:
            node = node.right
        else:
            return node
```

Recognition signal. For LCA + **BST**, don't reach for the general recursive algorithm first — ask "*can the ordering tell me which direction both nodes must be?*" If yes, an entire subtree is eliminated at every step, same as ordinary BST search.

8.3 Distance Between Two Nodes

Once we have the LCA, the path between `p` and `q` splits cleanly there:

```
dist(p, q) = dist(p, LCA) + dist(q, LCA)
            = depth(p) + depth(q) - 2 * depth(LCA)
```

```
def distance_between_nodes(root, p, q):
    """dist(p, q) = dist(p, LCA) + dist(q, LCA)."""
    ancestor = lca_binary_tree(root, p, q)

    def depth(node, target, d):
        if not node:
            return -1
        if node is target:
            return d
        left = depth(node.left, target, d + 1)
        if left != -1:
            return left
        return depth(node.right, target, d + 1)

    return depth(ancestor, p, 0) + depth(ancestor, q, 0)
```

Recognition signal. "*Find the distance between two nodes*" → find their LCA → measure each node's distance to it → add.

8.4 LCA with parent pointers — LeetCode 1650

Some variants give each node a `.parent` pointer instead of searching from the root. This becomes the same idea as **detecting where two linked lists intersect**: walk `p`'s ancestors into a set, then walk up from `q` until hitting a node already in that set.

```
def lca_with_parent_pointers(p, q):  
    """LeetCode 1650: nodes carry a `.parent` pointer instead. Walk p's ancestors into a set,  
    then walk up from q until we hit one - the two-pointer / linked-list-intersection idea."""  
    ancestors = set()  
    node = p  
    while node:  
        ancestors.add(node); node = node.parent  
    node = q  
    while node not in ancestors:  
        node = node.parent  
    return node
```

Part 9 · Pattern 6 — Path / Ancestor Problems

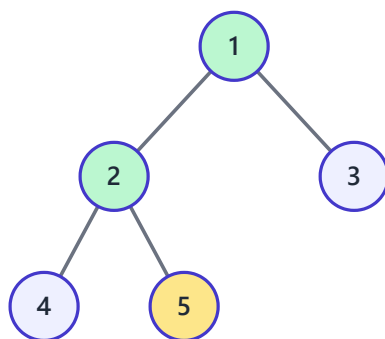
Recognition. The problem wants an actual **path** or the set of **ancestors**, not just a number. This is where `dfs(node)` first returns something closer to "did I find it?" than a numeric aggregate.

9.1 Root-to-Node Path

Return the sequence of nodes from the root to a given target. For target 5: $1 \rightarrow 2 \rightarrow 5$.

`dfs(node)` means "does the target exist somewhere in this subtree (including at `node` itself)?" If it does, `node` belongs on the path.

Root-to-node path: DFS reports "found?" and nodes append themselves on the way back up



`dfs(5)`: found itself → True, append 5

`dfs(2)`: right found → True, append 2

`dfs(1)`: left found → True, append 1

built backwards: $[5, 2, 1] \rightarrow \text{reverse} \rightarrow [1, 2, 5]$

There are two equally valid ways to build the path:

Discover on the way back up (the session's original version — the path is built **backwards** and needs reversing at the end):

call (indented)	result
5	IS the target → True, append
2	found below → True, append
1	found below → True, append

Built backwards as the recursion returns: $[5, 2, 1] \rightarrow \text{reversed} \rightarrow [1, 2, 5]$

```
def dfs(node):
    if not node:
        return False
    if node is target:
        path.append(node); return True
    if dfs(node.left) or dfs(node.right):
        path.append(node); return True
    return False
# path is now [target, ..., root] - reverse it
```


Backtrack on the way down (append before recursing, remove after — the same append/pop backtracking idiom used for Path Sum II in §5.2), which builds the path **forwards** and needs no reversal:

```
def root_to_node_path(root, target):
    """dfs(node) reports whether 'target' is in this subtree; on the way back up, nodes on the
    route append themselves (then the path is built in reverse and flipped)."""
    path = []

    def dfs(node):
        if not node:
            return False
        if node is target:
            path.append(node); return True
        if dfs(node.left) or dfs(node.right):
            path.append(node); return True
        return False

    dfs(root)
    return path[::-1]

def root_to_node_path_preorder(root, target):
    """The same path, built forward with explicit backtracking (append before recursing, pop
    after)."""
    path = []

    def dfs(node):
        if not node:
            return False
        path.append(node)
        if node is target or dfs(node.left) or dfs(node.right):
            return True
        path.pop()
        return False

    dfs(root)
    return path

def all_ancestors(root, target):
    """The root-to-node path, values only, excluding the target itself."""
    path = root_to_node_path_preorder(root, target)
    return [n.val for n in path[:-1]]
```

Recognition signal. *Root-to-node path, whether a subtree contains something, finding ancestors, path existence* → DFS where a subtree reports whether the target was found, and parents on the true branch reconstruct the path.

9.2 Path Between Two Nodes

Return the actual sequence of nodes between `p` and `q` (not just the distance).

Combine **two** patterns already built: **LCA** (where do the paths meet?) and **path reconstruction** (§9.1). The root-to-`p` and root-to-`q` paths share a prefix ending exactly at the LCA — reverse the `p`-side suffix (from the LCA to `p`) and append the `q`-side suffix (from the LCA to `q`):

```
def path_between(root, p, q):
    """LCA + path reconstruction: root-to-p and root-to-q share a prefix ending at the LCA;
    reverse the p side up to (and including) the LCA, then append the q side after it."""
    path_p = root_to_node_path_preorder(root, p)
    path_q = root_to_node_path_preorder(root, q)
    i = 0
    while i < len(path_p) and i < len(path_q) and path_p[i] is path_q[i]:
        i += 1
    return path_p[i - 1:][::-1] + path_q[i:]
```

Recognition signal. Asked for the actual path (not just its length) between two nodes → LCA + path-to-each-node + combine, as opposed to only needing the **distance**, which only needs the lengths (§8.3).

Part 10 · Tree Construction

Recognition. Given two traversal orders, rebuild the tree.

10.1 The structural insight

```
Preorder = ROOT → LEFT → RIGHT
Inorder  = LEFT → ROOT → RIGHT
Postorder = LEFT → RIGHT → ROOT
```

Preorder + Inorder → construct tree (LeetCode 105).

```
preorder = [1, 2, 4, 5, 3]
inorder  = [4, 2, 5, 1, 3]
```

The **first element of preorder is always the root** (nothing can come before ROOT in `ROOT → LEFT → RIGHT`). Finding that value in inorder splits it into `[4, 2, 5] | 1 | [3]` — everything before it is the left subtree, everything after is the right.

Construction: preorder gives the ROOT, inorder tells us where it SPLITS

preorder = [1, 2, 4, 5, 3] inorder = [4, 2, 5, 1, 3]



Preorder tells us WHO is the root. Inorder tells us WHICH nodes belong to the left/right subtree. Then the exact same reasoning applies recursively to the smaller pieces.

Finding the split without re-scanning inorder every time. Naively searching for each root's position costs $O(n)$ per call → $O(n^2)$ overall. Precompute `value → index in inorder` once, so every split is $O(1)$:

root taken from preorder	inorder window	split
1	inorder[0:5]	left = inorder[0:3], right = inorder[4:5]
2	inorder[0:3]	left = inorder[0:1], right = inorder[2:3]
4	inorder[0:1]	left = inorder[0:0], right = inorder[1:1]
5	inorder[2:3]	left = inorder[2:2], right = inorder[3:3]
3	inorder[4:5]	left = inorder[4:4], right = inorder[5:5]

```
def build_tree_pre_in(preorder_vals, inorder_vals):
    """LeetCode 105. Preorder's first element is always the root; inorder tells us which values
    belong left/right of it. A hashmap avoids re-scanning inorder for every root (O(n) not
    O(n^2))."""
    position = {v: i for i, v in enumerate(inorder_vals)}
    idx = 0

    def dfs(left, right):
        nonlocal idx
        if left > right:
            return None
        root_val = preorder_vals[idx]; idx += 1
        root = TreeNode(root_val)
        mid = position[root_val]
        root.left = dfs(left, mid - 1)
        root.right = dfs(mid + 1, right)
        return root

    return dfs(0, len(inorder_vals) - 1)
```

Complexity. $O(n)$ time, $O(n)$ space, thanks to the hashmap.

10.2 Inorder + Postorder → construct tree (LeetCode 106)

Postorder is `LEFT → RIGHT → ROOT`, so the **last** element is the root this time. We consume postorder from the end, and — because the last element of postorder belongs to whichever subtree was built *last* — we must build the **right** subtree before the **left**:

```
def build_tree_in_post(inorder_vals, postorder_vals):
    """LeetCode 106. Postorder's LAST element is the root; we consume postorder right to left,
    so the RIGHT subtree must be built before the LEFT."""
    position = {v: i for i, v in enumerate(inorder_vals)}
    idx = len(postorder_vals) - 1

    def dfs(left, right):
        nonlocal idx
        if left > right:
            return None
        root_val = postorder_vals[idx]; idx -= 1
        root = TreeNode(root_val)
        mid = position[root_val]
        root.right = dfs(mid + 1, right)
        root.left = dfs(left, mid - 1)
        return root

    return dfs(0, len(inorder_vals) - 1)
```

10.3 Preorder + Postorder → construct tree, full binary trees only (LeetCode 889)

Given only preorder and postorder, a tree is **not** uniquely determined in general (a node with exactly one child is ambiguous — is it the left or right child?). But if the tree is guaranteed **full** (every node has 0 or 2 children — see §3), the reconstruction is unique: `preorder[0]` is the root, and `preorder[1]` is the root of the left subtree; its position in postorder tells us exactly how many nodes the left subtree has.

```
def build_tree_pre_post(preorder_vals, postorder_vals):
    """LeetCode 889 (full binary tree: every node has 0 or 2 children, so this is UNIQUE).
    preorder[0] is the root; preorder[1] is the root of the LEFT subtree, and its position in
    postorder tells us how big the left subtree is."""
    position = {v: i for i, v in enumerate(postorder_vals)}

    def dfs(pre_l, pre_r, post_l, post_r):
        if pre_l > pre_r:
            return None
        root = TreeNode(preorder_vals[pre_l])
        if pre_l == pre_r:
            return root
        left_root_val = preorder_vals[pre_l + 1]
        left_size = position[left_root_val] - post_l + 1
        root.left = dfs(pre_l + 1, pre_l + left_size, post_l, post_l + left_size - 1)
        root.right = dfs(pre_l + left_size + 1, pre_r, post_l + left_size, post_r - 1)
        return root

    return dfs(0, len(preorder_vals) - 1, 0, len(postorder_vals) - 1)
```

10.4 Sorted Array to BST (LeetCode 108)

Not a two-traversal reconstruction, but the same "pick a natural root, then recurse on the pieces" idea: with values already sorted, the **middle** element is the natural, balance-preserving root (already used in §7.6 to balance an existing BST).

```
def sorted_array_to_bst(nums):
    """LeetCode 108: the same 'pick the middle as root' idea as balance_bst, from a sorted array
    directly."""
    def build(lo, hi):
        if lo > hi:
            return None
        mid = (lo + hi) // 2
        node = TreeNode(nums[mid])
        node.left = build(lo, mid - 1)
        node.right = build(mid + 1, hi)
        return node

    return build(0, len(nums) - 1)
```


Part 11 · Serialization / Deserialization

Recognition. Store a tree so it can later be rebuilt exactly.

11.1 Why values alone aren't enough

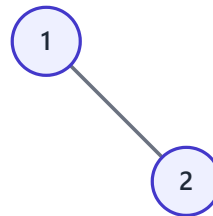
```
  1      1
 /      \
2        2
```

These are different trees, but if we only store the values 1, 2 we can't tell them apart. **Missing children must be represented too**, with a marker such as #.

Serialization: a # marker for every MISSING child keeps the structure unambiguous



left-child chain



right-child chain

values only: both give "1,2" — ambiguous!

with # markers: "1,2,##,##" vs "1,2,##,##" — each tree is unique

1,2,##,## (left-child chain) vs. 1,2,##,## (right-child chain) — including # for every missing child makes the sequence **unambiguous**: every real node and every missing child gets exactly one symbol.

11.2 Recursive (preorder) codec — LeetCode 297

Tree [1, 2, 3, null, null, 4] serializes (preorder, # = missing child) to:

```
1,2,##,3,4,##,##
```

Deserializing that string reconstructs the exact same tree: [1, 2, 3, None, None, 4].

```

class Codec:
    """LeetCode 297. Preorder with a '#' marker for every missing child: this makes the encoding
    unambiguous (a value alone can't distinguish a left-only tree from a right-only one)."""

    def serialize(self, root):
        out = []

        def dfs(node):
            if not node:
                out.append("#"); return
            out.append(str(node.val))
            dfs(node.left); dfs(node.right)

        dfs(root)
        return ",".join(out)

    def deserialize(self, data):
        values = iter(data.split(","))

        def dfs():
            v = next(values)
            if v == "#":
                return None
            node = TreeNode(int(v))
            node.left = dfs()
            node.right = dfs()
            return node

        return dfs()

```

11.3 Iterative variants

The harder part isn't the idea — it's keeping the **same structural ordering** without the call stack doing the bookkeeping for you. The serializer reuses the reverse-push trick from §2.3 (push right, then left, so left is processed first):


```

def serialize_iterative(root):
    """The iterative twin: same reverse-push trick as iterative preorder (push right, then left)."""
    if not root:
        return "#"
    out, stack = [], [root]
    while stack:
        node = stack.pop()
        if node is None:
            out.append("#"); continue
        out.append(str(node.val))
        stack.append(node.right)
        stack.append(node.left)
    return ",".join(out)

def deserialize_iterative(data):
    """Rebuild from the iterative-preorder string: every node needs its left, then its right, filled in."""
    values = data.split(",")
    if values[0] == "#":
        return None
    root = TreeNode(int(values[0]))
    stack = [(root, "left")]
    i = 1
    while stack and i < len(values):
        node, side = stack[-1]
        v = values[i]; i += 1
        if v == "#":
            if side == "left":
                stack[-1] = (node, "right")
            else:
                stack.pop()
        else:
            child = TreeNode(int(v))
            setattr(node, side, child)
            if side == "left":
                stack[-1] = (node, "right")
            else:
                stack.pop()
            stack.append((child, "left"))
    return root

```

A BFS-based (level-order) codec is also common on online judges and works the same way, one queue instead of one stack:

```

def serialize_level_order(root):
    """A BFS-based codec (used by many online judges), including '#' for missing children."""
    if not root:
        return "#"
    out, q = [], deque([root])
    while q:
        node = q.popleft()
        if node is None:
            out.append("#"); continue
        out.append(str(node.val))
        q.append(node.left)
        q.append(node.right)
    return ",".join(out)

def deserialize_level_order(data):
    values = data.split(",")
    if values[0] == "#":
        return None
    root = TreeNode(int(values[0]))
    q = deque([root])
    i = 1
    while q:
        node = q.popleft()
        if i < len(values):
            v = values[i]; i += 1
            if v != "#":
                node.left = TreeNode(int(v)); q.append(node.left)
        if i < len(values):
            v = values[i]; i += 1
            if v != "#":
                node.right = TreeNode(int(v)); q.append(node.right)
    return root

```

Part 12 · Tree DP — Multiple Subtree States

Recognition. The parent's decision about the current node changes what its children are allowed to contribute. One number returned per subtree is no longer enough.

```
def dfs(node):
    if not node:
        return BASE_STATE
    left = dfs(node.left)
    right = dfs(node.right)
    state1 = ...
    state2 = ...
    return state1, state2
```

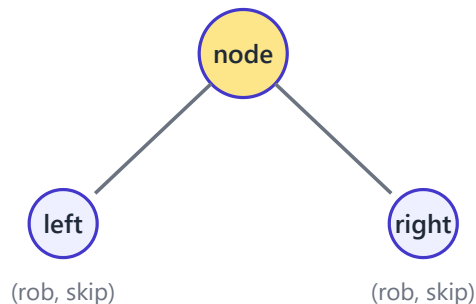
12.1 House Robber III (LeetCode 337)

Rob a tree of houses; robbing a node forbids robbing its **direct children**. Maximise the total.

We can't just take `max(subtree)` independently per side, because whether we rob the current node changes what its children are allowed to give us. So `dfs(node)` returns **two** numbers:

```
rob(node) = best total if we ROB this node    → we CANNOT rob either child
skip(node) = best total if we DON'T rob it    → each child is free to be robbed or skipped
```

Tree DP (House Robber III): `dfs(node)` returns TWO states, not one



$\text{rob}(\text{node}) = \text{node.val} + \text{left.skip} + \text{right.skip}$ (children can't also be robbed)

$\text{skip}(\text{node}) = \max(\text{left.rob}, \text{left.skip}) + \max(\text{right.rob}, \text{right.skip})$

Deriving the two branches. If we rob `node`, we must take `skip` from both children (never `rob`): $\text{rob}(\text{node}) = \text{node.val} + \text{left.skip} + \text{right.skip}$. If we don't rob `node`, each child independently picks whichever is larger: $\text{skip}(\text{node}) = \max(\text{left.rob}, \text{left.skip}) + \max(\text{right.rob}, \text{right.skip})$.

node	left state	right state	rob(node)	skip(node)
3	(left) rob=0,skip=0	(right) rob=0,skip=0	rob=3	skip=0
2	(left) rob=0,skip=0	(right) rob=3,skip=0	rob=2	skip=3

node	left state	right state	rob(node)	skip(node)
1	(left) rob=0,skip=0	(right) rob=0,skip=0	rob=1	skip=0
3	(left) rob=0,skip=0	(right) rob=1,skip=0	rob=3	skip=1
3	(left) rob=2,skip=3	(right) rob=3,skip=1	rob=7	skip=6

Answer = max(rob(root), skip(root)) = **7**

```
def rob(root):
    """LeetCode 337. dfs(node) -> (rob, skip): robbing this node forces skip on both children;
    skipping this node lets each child be independently robbed or skipped."""
    def dfs(node):
        if not node:
            return 0, 0
        l_rob, l_skip = dfs(node.left)
        r_rob, r_skip = dfs(node.right)
        rob_here = node.val + l_skip + r_skip
        skip_here = max(l_rob, l_skip) + max(r_rob, r_skip)
        return rob_here, skip_here

    return max(dfs(root))
```

The final answer is `max(rob(root), skip(root))`.

Recognition signal. "Your decision at a node changes what is allowed at its children" → multiple states, bottom-up DFS.

12.2 Binary Tree Cameras (LeetCode 968)

Place the fewest cameras (on nodes) so that every node is covered — a camera covers itself, its parent, and its children.

This is harder than House Robber III because the states aren't a simple "take/don't take" pair — they describe **coverage relationships**, and the choice is made greedily bottom-up rather than by comparing two totals. Three states per node:

```
0 = not covered          (needs help from its parent)
1 = has a camera         (covers itself, its parent, its children)
2 = covered, no camera   (covered by a child's camera)
```

`None` counts as **covered** (state `2`) — otherwise every leaf would look "uncovered" by its non-existent children and be forced into placing a camera on itself.

The greedy rule at each node, evaluated bottom-up so a decision is only made once both children's states are already known: if **either child is uncovered (0)**, the current node **must** get a camera (state `1`) to cover that child — right now is the last chance, since we're heading up and away from that child. Otherwise, if either child **has** a camera (1), the current node is covered by it (state `2`). Otherwise both children are merely *covered* (2), so the current node itself is **not yet covered** (state `0`), leaving the decision to its parent.

```
def min_camera_cover(root):
    """LeetCode 968. Three states per node: 0 = uncovered, 1 = has a camera, 2 = covered (by a
    child's camera). None counts as covered (2) so leaves don't force a camera onto themselves."""
    cameras = 0

    def dfs(node):
        nonlocal cameras
        if not node:
            return 2
        left = dfs(node.left)
        right = dfs(node.right)
        if left == 0 or right == 0:
            cameras += 1
            return 1
        if left == 1 or right == 1:
            return 2
        return 0

    if dfs(root) == 0:
        cameras += 1
    return cameras
```

If the **root** itself comes back as state `0` (uncovered, with no parent left to place a camera), one final camera is added for it.

Why the greedy choice is optimal (brief intuition). Placing a camera at the *lowest* possible node that can cover an uncovered child is never worse than placing it higher: a lower camera still covers that child (and the node itself, and the node's parent), while a higher placement risks leaving the child uncovered. This is the same "resolve constraints as low in the tree as possible" idea used, in a different guise, throughout Part 4's BST deletion.

12.3 Largest BST Subtree (Mixed Hard 1)

Find the size of the largest subtree that is itself a valid BST.

The parent needs **four** pieces of information about each child's subtree: is it a BST, its minimum value, its maximum value, and its size. A node forms a BST if and only if both children already are BSTs **and** `left.max < node.val < right.min` — the same range-propagation idea as `is_valid_bst` (§7.4), but derived bottom-up instead of top-down because we don't know the eventual global bounds in advance.

```

def largest_bst_subtree(root):
    """Mixed Hard 1. dfs(node) -> (is_bst, min_val, max_val, size). A node forms a BST only if
    both children already are BSTs AND left.max < node.val < right.min."""
    answer = 0

    def dfs(node):
        nonlocal answer
        if not node:
            return True, float("inf"), float("-inf"), 0
        l_bst, l_min, l_max, l_size = dfs(node.left)
        r_bst, r_min, r_max, r_size = dfs(node.right)
        if l_bst and r_bst and l_max < node.val < r_min:
            size = l_size + r_size + 1
            answer = max(answer, size)
            return True, min(l_min, node.val), max(r_max, node.val), size
        return False, 0, 0, 0

    dfs(root)
    return answer

```

This combines **the BST invariant + bottom-up DFS + a multi-field subtree summary + a global answer** — every major idea from Parts 6, 7 and 12 in one problem, which is exactly the kind of combination the session's mixed-problem phase calls for.

Part 13 · Mixed / Hard Problems — Choosing the Right Tool

The final skill isn't another tree-specific trick — it's recognising when a tree problem is really **a different kind of problem that happens to live on a tree**.

13.1 Choose BFS, not DFS, for "nearest"

Find the first node at **minimum depth** with a given value.

Minimum depth / nearest is the BFS recognition signal from the Queue guide, applied here: the **first** match BFS finds is guaranteed to be at the shallowest depth, because BFS explores depth `0`, then `1`, then `2`, in order.

```
def nearest_value_bfs(root, x):
    """Hard Mixed 2: the FIRST node with value x found by BFS is guaranteed to be at minimum
    depth."""
    if not root:
        return -1
    q = deque([(root, 0)])
    while q:
        node, depth = q.popleft()
        if node.val == x:
            return depth
        if node.left:
            q.append((node.left, depth + 1))
        if node.right:
            q.append((node.right, depth + 1))
    return -1
```

13.2 Tree + prefix-sum hashmap

Count downward paths (not necessarily starting at the root) that sum to `k` — LeetCode 437, *Path Sum III*.

A plain root-to-leaf accumulated sum isn't enough, because the path can **start anywhere**. This is the array prefix-sum + hashmap trick ("if `current_sum - old_sum = k`, the nodes between them sum to `k`"), adapted to a tree with **backtracking on the map** so a value doesn't leak into unrelated branches:

```

def path_sum_iii(root, k):
    """LeetCode 437: count downward paths (not necessarily from the root) that sum to k, using the
    prefix-sum + hashmap trick from arrays, adapted to a tree with backtracking on the map."""
    prefix = {0: 1}
    count = 0

    def dfs(node, current_sum):
        nonlocal count
        if not node:
            return
        current_sum += node.val
        count += prefix.get(current_sum - k, 0)
        prefix[current_sum] = prefix.get(current_sum, 0) + 1
        dfs(node.left, current_sum)
        dfs(node.right, current_sum)
        prefix[current_sum] -= 1
        if prefix[current_sum] == 0:
            del prefix[current_sum]

    dfs(root, 0)
    return count

```

The lesson. *Tree problem $\neq$ tree-specific trick only.* The underlying problem here is *root-to-current path + prefix sums + hashmap* — the tree is just the shape it's embedded in.

13.3 Tree + heap

Find the k largest values in a binary tree.

No tree-specific reasoning needed at all: traverse every node (any order), and maintain the k largest seen so far with a **min-heap of size k** (the Heap guide's Top-K pattern).


```

def k_largest_values(root, k):
    """Hard Mixed 4: the tree is just what we traverse; the actual selection is a min-heap of size
    k."""
    heap = []

    def dfs(node):
        if not node:
            return
        heapq.heappush(heap, node.val)
        if len(heap) > k:
            heapq.heappop(heap)
        dfs(node.left)
        dfs(node.right)

    dfs(root)
    return sorted(heap, reverse=True)

```

13.4 Tree + coordinates (top-down state through a queue)

Vertical Order Traversal groups nodes by their horizontal position. This is the *BFS + positional state* idea from Maximum Width (§2.6), generalised: carry a `(row, col)` coordinate with each queued node, then group and sort by column.

```

def vertical_order(root):
    """LeetCode 987-style vertical order: BFS with a (row, col) coordinate per node (top-down state
    again, this time carried through a queue instead of recursion), grouped and sorted by column."""
    if not root:
        return []
    cols = {}
    q = deque([(root, 0, 0)])
    while q:
        node, row, col = q.popleft()
        cols.setdefault(col, []).append((row, node.val))
        if node.left:
            q.append((node.left, row + 1, col - 1))
        if node.right:
            q.append((node.right, row + 1, col + 1))
    result = []
    for col in sorted(cols):
        vals = [v for _, v in sorted(cols[col], key=lambda rv: (rv[0], rv[1]))]
        result.append(vals)
    return result

```

Boundary Traversal stitches together three small variations of the traversal idea already covered: the **left boundary** walked top-down, the **leaves** in any left-to-right order, and the **right boundary** walked top-down but appended in reverse (bottom-up) — one pass, three sub-traversals.

```

def boundary_traversal(root):
    """LeetCode 545-style: left boundary (top-down) + leaves (any order) + right boundary reversed
    (bottom-up), each a small variation of the same DFS idea, stitched into one pass."""
    if not root:
        return []
    result = [root.val] if root.left or root.right else []

    def left_boundary(node):
        while node:
            if node.left or node.right:
                result.append(node.val)
            node = node.left if node.left else node.right

    def leaves(node):
        if not node:
            return
        if not node.left and not node.right:
            result.append(node.val)
            return
        leaves(node.left)
        leaves(node.right)

    def right_boundary(node):
        stack = []
        while node:
            if node.left or node.right:
                stack.append(node.val)
            node = node.right if node.right else node.left
        result.extend(reversed(stack))

    left_boundary(root.left)
    leaves(root)
    right_boundary(root.right)
    return result

```

13.5 The tree pattern map

Problem signal	Think
Need a particular node order	DFS traversal — preorder/inorder/postorder
Level-by-level, minimum edges	BFS
Parent needs child information	Bottom-up DFS
Root/path information needed at a node	Top-down DFS
Best answer anywhere in the tree	DFS + global answer
Any-node → any-node path	usually bottom-up + global answer

Problem signal	Think
Root → leaf constraint	Top-down / path DFS
Given a BST	exploit the ordering
Kth smallest in a BST	inorder
BST validation	range propagation (top-down state)
Two target nodes, common ancestor	LCA
BST + LCA	exploit ordering, skip the general algorithm
Reconstruct a tree from traversals	preorder/postorder gives the root, inorder splits left/right
Preserve exact structure (store/transmit)	serialization, with a marker for missing children
A node's decision affects its children	Tree DP
Multiple subtree states needed	Tree DP
Level <i>positions</i> matter (not just values)	BFS + positional state
Tree + top-k	heap
Tree + path-sum anywhere	DFS + prefix-sum hashmap

Part 14 · Templates, Complexity and the Problem-Solving Checklist

14.1 The four templates to recognise, not memorise

Bottom-up — *what does my subtree tell my parent?*

```
def dfs(node):
    if not node:
        return BASE
    left = dfs(node.left)
    right = dfs(node.right)
    return COMBINE(left, right)
```

Top-down — *what does my parent tell me?*

```
def dfs(node, state):
    if not node:
        return
    new_state = UPDATE(state, node)
    dfs(node.left, new_state)
    dfs(node.right, new_state)
```

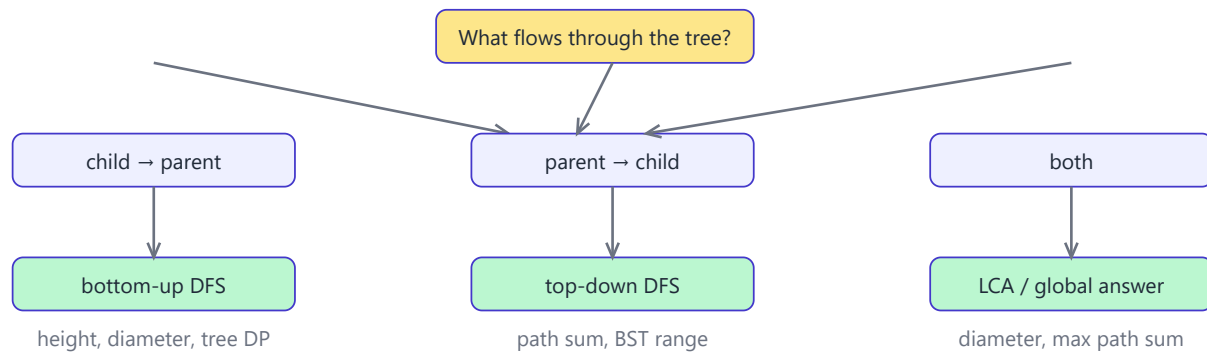
Global answer — *is the answer different from what my parent needs from me?*

```
answer = ...
def dfs(node):
    if not node:
        return BASE
    left = dfs(node.left)
    right = dfs(node.right)
    candidate = ...
    answer = max(answer, candidate)
    return information_for_parent
```

Tree DP — *does the parent need multiple possible states of my subtree?*

```
def dfs(node):
    if not node:
        return BASE_STATE
    left = dfs(node.left)
    right = dfs(node.right)
    state1 = ...
    state2 = ...
    return state1, state2
```

The deeper picture: what does the parent need FROM the children?



14.2 The complexity rule

For every recursive tree solution, keep two variables straight:

n = number of nodes h = height

A normal DFS visits each node once → **time** = $O(n)$. The recursive call stack holds at most one frame per level of depth → **space** = $O(h)$.

balanced tree: $h = O(\log n)$

skewed tree: $h = O(n)$

Don't say "recursive DFS is $O(\log n)$ space." The correct general statement is $O(h)$ — and a tree built from sorted or adversarial input can be fully skewed, making $h = n$. This is exactly why §3's structural classifications matter for complexity, not just terminology.

14.3 The actual problem-solving procedure

When you meet a new tree problem, run this checklist rather than pattern-matching a name:

1. What is the problem actually asking?
2. Do I care about **order**? **levels**? **paths**? **subtree properties**? **BST ordering**?
3. DFS or BFS?
4. If DFS: **top-down** or **bottom-up**?
5. What does `dfs(node)` **mean**?
6. What should `dfs(node)` **return**?
7. What do I get from `left`? From `right`?
8. How do I **combine** them?
9. Is the final answer the value **returned by** `dfs(root)`, or do I need a **separate global answer**?
10. What **invariant** must always remain true?
11. What is the $O(n)$ / $O(h)$ cost?
12. Could the tree be **skewed**?

This — not memorising `diameter()`, `maxPathSum()`, `lca()` by name — is the actual skill. The flow is always: **information direction** → **what `dfs(node)` means** → **what it returns** → **whether a global answer is needed** → **the invariant**.

14.4 Pitfalls checklist

Pitfall	Fix
<code>dfs(None) = 0</code> for edge-based height	Derive the base case from the recurrence — it must be <code>-1</code>
Checking only <code>left.val < node.val < right.val</code> for BST validity	Carry the full <code>(low, high)</code> range , not just the parent comparison
<code>diameter = left + right + 1</code>	It's <code>+2</code> — one edge into each side (§6.1)
Returning <code>combine(left, right)</code> for the parent when a path can only extend through one side	Distinguish <i>return value</i> (one side) from <i>global candidate</i> (both sides)
Treating "path" problems as automatically top-down	Ask whether the parent needs a child's result (bottom-up) instead
One number returned when the parent's decision depends on multiple states	Tree DP — return a tuple
Forgetting that <code>None</code> needs an explicit meaning in serialization	Store a marker (<code>#</code>) for every missing child
Assuming a recursive tree algorithm is <code>O(log n)</code> space	It's <code>O(h)</code> ; a skewed tree makes <code>h = O(n)</code>
Overusing structural classifications (full/complete/perfect) as the main tool	They're extra information about the input, not a substitute for the checklist above

14.5 Which patterns combine?

- **BST + range propagation + global answer** — Largest BST Subtree (§12.3).
- **LCA + path reconstruction** — Path Between Two Nodes (§9.2).
- **Bottom-up + global answer**, twice over — Diameter (§6.1) and Longest Univalued Path (§4.6) are the *same* structural idea at two different points in the guide.
- **Top-down state as a constraint** (BST validation, §7.4) vs. **top-down state as an accumulator** (path sum, §5.1) — same mechanism, different purpose.
- **BFS + positional state** — Maximum Width (§2.6) and Vertical Order (§13.4).
- **Tree DP with 2 states** (House Robber III) vs **3 states** (Binary Tree Cameras) — §12.1–12.2.
- **Tree traversal + an unrelated data structure** — heap (§13.3), hashmap (§13.2) — the tree supplies the *order to visit nodes in*, nothing more.

Part 15 · Practice, Recognition Drills and Interview Q&A

15.1 Practice list (LeetCode)

Pattern	Start with	Then	Hard
Traversals	Binary Tree Preorder/Inorder/Postorder (144/94/145)	Binary Tree Level Order Traversal (102)	N-ary Tree Postorder, iterative variants
Bottom-up DFS	Maximum Depth (104) · Same Tree (100)	Balanced Binary Tree (110) · Invert Binary Tree (226) · Subtree of Another Tree (572)	Longest Univalue Path (687)
Top-down DFS	Path Sum (112)	Path Sum II (113) · Sum Root to Leaf Numbers (129) · Count Good Nodes (1448)	Maximum Width of Binary Tree (662)
Path + global answer	Diameter of Binary Tree (543)	—	Binary Tree Maximum Path Sum (124)
BST	Search in a BST (700) · Validate BST (98)	Kth Smallest in a BST (230) · Insert into a BST (701)	Delete Node in a BST (450) · Balance a BST (1382)
LCA	LCA of a BST (235)	LCA of a Binary Tree (236)	LCA of Deepest Leaves (1123)
Path/ancestor	Root-to-node path (custom)	—	Path Between Two Nodes / Distance Between Nodes
Construction	Construct from Preorder & Inorder (105)	Construct from Inorder & Postorder (106)	Construct from Preorder & Postorder (889)
Serialization	Serialize and Deserialize Binary Tree (297)	—	iterative / BFS-based codec
Tree DP	House Robber III (337)	—	Binary Tree Cameras (968) · Largest BST Subtree
Mixed	k Largest Values (custom)	Path Sum III (437)	Vertical Order Traversal (987) · Boundary of Binary Tree (545)

15.2 Recognition drills

#	Statement	Answer
1	"Return the values level by level."	BFS ; <code>level_size = len(queue)</code> freezes each level
2	"The maximum value anywhere in the tree."	Bottom-up DFS ; <code>dfs(node) = max(node.val, left, right)</code>
3	"Does a root-to-leaf path sum to X?"	Top-down DFS ; carry <code>current_sum</code>
4	"The longest path between any two nodes."	Bottom-up + global answer (diameter pattern)

#	Statement	Answer
5	"Kth smallest value in a BST."	BST → inorder → sorted → kth element
6	"Is every node's value within its ancestors' range?"	Top-down DFS with a (low, high) state
7	"Where do the paths to two given nodes first meet?"	LCA
8	"Rebuild the tree from preorder and inorder."	preorder gives the root, inorder splits left/right , recurse
9	"My decision at this node restricts what my children may do."	Tree DP — return multiple states
10	"First node at minimum depth with value X."	BFS , not DFS — first arrival is shallowest
11	"Downward paths (any start) summing to k."	Tree + prefix-sum hashmap
12	"k largest values in the tree."	Tree traversal + a min-heap of size k

15.3 Interview questions and short answers

Why does recursion fit trees so naturally? Every node is simultaneously part of the recursion *and* the root of its own smaller tree, so `solve(node)` can be defined purely in terms of `solve(node.left)` and `solve(node.right)`.

What's the difference between depth and height? Depth looks toward the root (how far *up*); height looks toward the leaves (how far *down*). `dfs(None) = -1` for edge-based height so a leaf naturally computes to `0`.

How are preorder, inorder and postorder related to recursion patterns? They are the same recursive skeleton with `PROCESS(node)` in a different position. Postorder (`left, right, then me`) is exactly what bottom-up DFS does, because the parent can only combine its children's answers *after* they return.

Why does iterative preorder push the right child before the left? The stack is LIFO; to pop the left child first, it must go on top, so it's pushed *last*.

Why is BST validation not just `left.val < node.val < right.val`? A node must satisfy **every** ancestor's constraint, not only its immediate parent's — a deep node could locally look fine while violating a bound set several levels up. Carry a `(low, high)` range down instead.

Why does diameter need `left + right + 2`, not `+1`? With height measured in edges, connecting a left subtree of height `left` and a right subtree of height `right` through the current node uses **two** edges — one from each side into the node — on top of the `left` and `right` edges already inside each subtree.

What is the difference between what `dfs(node)` returns and the "global answer" in diameter/max-path-sum problems? The **return value** can only represent one branch, because the parent can only extend the path through one side without forking it. The **global answer** may use both sides, since it represents a path that terminates at the current node rather than continuing upward.

Why does LCA return a node (or None) instead of a number? Because `dfs(node)`'s job here is to report *what was found* in this subtree — which could be nothing, one target, or the LCA itself — not a numeric aggregate. `dfs` can return anything the parent needs.

Why is Tree DP different from ordinary bottom-up DFS? The parent's own decision (rob vs. skip; camera vs. no camera) determines *which* of the children's states are usable, so one number per subtree is not enough — the recursion must return multiple states.

What is the actual time/space complexity of a recursive tree traversal? Time is $O(n)$ (every node visited once); space is $O(h)$, the recursion depth — $O(\log n)$ for a balanced tree, but $O(n)$ in the worst (skewed) case.

When should you prefer BFS over DFS on a tree? Whenever the question involves *levels*, *minimum depth*, or *"the nearest X"* — BFS explores in non-decreasing depth order, so the first match is guaranteed to be the shallowest.

15.4 One-page recap

TREE a node is both part of the recursion AND the root of its own subtree
dfs(node): what does it MEAN? what does it RETURN?

TRAVERSAL preorder = me->L->R inorder = L->me->R postorder = L->R->me
postorder IS bottom-up DFS: the parent needs its children's answers first

ITERATIVE recursion = an implicit call stack; push the LATER child first

BFS level_size = len(queue) freezes one level at a time

BOTTOM-UP child -> parent dfs(node) returns subtree info

TOP-DOWN parent -> child dfs(node, state) carries state down

GLOBAL candidate may use BOTH sides; the RETURN VALUE uses only ONE

TREE DP the current node's decision restricts the children -> return multiple states

BST ordering eliminates a whole subtree per step; inorder = sorted

VALIDATE carry a (low, high) RANGE down, not just a parent comparison

LCA both sides found -> I'm the LCA; one side found -> pass it up
BST: ordering picks the direction directly

CONSTRUCTION preorder/postorder gives the ROOT; inorder SPLITS left/right

SERIALIZE a marker (#) for every missing child keeps it unambiguous

COMPLEXITY time $O(n)$, space $O(h)$; $h = O(\log n)$ balanced, $O(n)$ skewed

ASK what does dfs(node) mean? what does it return? bottom-up or top-down?
DFS or BFS? is there a global answer? what invariant holds?